KINGDOM OF SAUDI ARABIA
Ministry of Higher Education
Taibah University
College of Computer Science and
Engineering (Girls Section)

جامعة طيبة
TAIBAH UNIVERSITY

المملكة العربية السعودية
وزارة التعليم العالي
جامعة طيبة
كلية علوم وهندسة الحاسب الآلي

Music Genre Classification

A Project Report Submitted to Fulfill the Requirements of AI331

Section F3

SID	Student Name	Responsibilities
4454243	Abeer Anwar Alansari	1. UMAP 2. HBDSCAN
4453267	Bashayer Abdulaziz Alharbi	1. Preprocessing 2. FNN Model
4451941	Shumukh Eid Alotaibi	1. Preprocessing 2. EDA 3. Xgboost Model
4454147	Hatoon Mohammed Ghabben	1. Preprocessing 2. UMAP 3. k-Means Clustering 4. Multistage Clustering
4451274	Layal Mohammed Alhazmi	1. Preprocessing 2. EDA 3. Random Forest Model 4. Catboost Model

Supervised By: Dr. Samah Aloufi

13 December 2025

Abstract

The rapid growth of digital music platforms has increased demand for automated methods to analyze and organize music by acoustic similarity. Metadata-based approaches are limited, so machine learning will extract meaningful patterns directly from audio data. This study investigates algorithms and learning methods for music analysis using the Free Music Archive (FMA) dataset, which contains high-dimensional features extracted from musical tracks.

In the supervised learning setting, several tree-based classification models were evaluated for genre prediction. Random Forest was used as a baseline ensemble method based on bagging, while CatBoost and XGBoost were applied as boosting-based models to capture more complex feature interactions. Feedforward Neural Network (FNN) used to explore hidden patterns and discover bright solution that lead us to model creation with better insights.

For unsupervised analysis, clustering techniques were applied to explore similarities in data without relying on data labels. The evaluated methods included K-Means, HDBSCAN, and a multistage clustering combining HDBSCAN with K-Means.

Overall, XGBoost was the most effective model for supervised music genre classification outperforming the other supervised approaches. And with HDBSCAN, it achieved the best clustering performance among the evaluated methods.

.

Keywords: FMA Dataset, Machine Learning, Music Genre Classification.

Table of Content

Chapter 1: Introduction

1.1 Problem Motivation and Domain

Musical data analysis is a central focus in the field of music information extraction (MIR). It combines digital data processing, artificial intelligence, and machine learning to understand sonic structure automatically. This field aims to develop methods that enable humans to comprehend music by studying the acoustic properties derived from numerical representations, which reflect multiple aspects of sound, including spectral structure, rhythmic characteristics, and temporal patterns. These interact to represent the identity of a musical work in a digitally analyzable way.

Musical data differs from other sonic data because it contains multiple structural levels: the raw signal level, the geometric features level, the musical patterns level, and finally, the human classification level. This multi-layered nature makes music high-dimensional and non-linear data, requiring advanced methods capable of automatically detecting structures, analyzing them, extracting features, and classifying or grouping them based on sonic similarity.

Given the vast diversity of musical works and the overlap of their characteristics, manual or rule-based analysis is insufficient to meet the processing demands and high accuracy required to represent the deep sonic structure of songs. This has increased the need for precise music analysis techniques based on MIR, which necessitates the use of machine learning methods as the most effective way to discover hidden patterns and sonic clusters and identify subtle similarities between sonic segments.

1.2 Objectives and Scope

This project focuses on analyzing high-dimensional musical data to understand and discover underlying patterns within the feature space extracted from **Free Music Archive dataset** The project's focus is on evaluating the results of supervised learning models, Random Forest, CatBoost and XGBoost which are used to predict musical genres using metadata and features. It also includes comparing the results of unsupervised learning models, K-Means Clustering, HDBSCAN Clustering, and Multi-Stage Clustering with classifications of musical genres and determining whether the discovered sonic clusters truly reflect genre boundaries or reveal new, undocumented patterns. The feedforward neural network (FNN) was included as another supervised model to compare with the classifiers. Unlike Random Forest, CatBoost, and XGBoost, which rely on decision trees, the FNN learns directly from the feature values through layers of neurons.

Chapter 2: Dataset and Domain Description

2.1 Dataset Overview

The Dataset used in this project is the **Free Music Archive dataset**, developed for music analysis and machine learning research, it contains 106574 music tracks, which makes it suitable for large scale and high dimensional data analysis.

In this project, three files from the dataset were used:

- **Tracks.csv:** provides metadata for each track, including track ID, artist info, and genre labels.
- **Genre.csv:** contains info about music genres and their hierarchical structure, it helps in extract target label.
- **Features.csv:** contains preprocessed audio features

The dataset has many features as each sample represents a single music track while the target is a categorical genre label. Standard preprocessing techniques were applied to ensure the data was suitable for model training.

2.2 Dataset Complexity and Challenge

The dataset faces several challenges that increase its overall complexity. First, the dataset is high dimensional, with 518 features per track, which increase computational cost.

Second, the dataset suffers from class imbalance, as some music genres are represented by significantly more samples than others. This imbalance can bias model performance toward majority classes.

Another important challenge is genre overlapping. Some music tracks belong to multiple genres. This overlap increases classification ambiguity and can negatively affect model performance.

Additionally, the noisy nature of audio data and the large dataset size increased computational requirements, leading to the use of Google Colab GPUs.

Chapter 3: Theoretical Background

3.1 Supervised Algorithm (Random Forest Classifier)

Random Forest (RF) is a supervised ensemble learning algorithm that works by combining predictions from a pre-specified number of decision trees. It is based on two main principles: random sampling (bagging), where each decision tree is trained on a subset of the training samples, and feature subset selection, where each tree uses a random subset of features. The goal of these two sources of randomness is to reduce the variance of the estimator, as individual decision trees tend to exhibit high variance and are often prone to overfitting. By averaging these separate predictions, the model achieves better generalization. The final class prediction is determined by majority voting from the individual classifiers. Mathematically, this can be expressed as maximizing the likelihood estimate through aggregation:

$$\hat{y} = \arg\max_{k} \sum_{b=1}^{B} 1(h_{b(x)} = k)$$

3.2 Supervised Algorithm (CatBoost Classifier)

CatBoost (boosting) is based on the concept of gradient boosting technique where decision trees are built sequentially to minimize errors and improve predictions. The process works by constructing a decision tree and evaluating how many errors there are in predictions. Once the first tree is built the next tree is created to correct the errors made by the previous one. This process continues iteratively with each new tree focusing on improving the model's predictions by reducing previous errors this process continues till a predefined number of iterations met. The result is an ensemble of decision trees that work together to provide accurate predictions.

It is particularly well-suited for large-scale datasets with many independent features.

3.3 Supervised Algorithm (XGBoost Classifier)

XGBoost (eXtreme Gradient Boosting) is a supervised ensemble learning algorithm based on gradient boosting, designed for classification and regression problems. The model is trained in a stage-wise additive manner, where weak learners in the form of decision trees are sequentially added to minimize a predefined loss function. The learning process begins by initializing the model with a constant prediction that minimizes the overall loss across the training data:

$$\hat{f}^{(0)}(x) = \arg\min_{\theta} \sum_{i=1}^{N} L(y_i, \theta)$$

This initial model serves as a baseline upon which subsequent trees are added. At each boosting iteration m, XGBoost computes the first-order gradient and second-order Hessian of the loss function with respect to the current model predictions:

$$\hat{g}_m(x_i) = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x)=\hat{f}(m-1)_{(x)}}, \hat{h}_m(x_i) = \left[\frac{\partial^2 L(y_i, f(x_i))}{\partial f(x_i)^2}\right]_{f(x)=\hat{f}(m-1)_{(x)}}$$

Using a second-order Taylor approximation of the loss function, the optimization problem of learning the next weak learner is transformed into a weighted least squares problem. The new tree ϕ_m is obtained by solving:

$$\hat{\phi}_m = arg \min_{\phi \in \Phi} \sum_{i=1}^{N} \frac{1}{2} \hat{h}_m(x_i) \left[-\frac{\hat{g}_m(x_i)}{\hat{h}_m(x_i)} - \phi_m(x_i)\right]^2$$

The output of the newly learned tree is then scaled by a learning rate α, which controls the contribution of each tree to the overall model:

$$\hat{p}_m(x) = \alpha \hat{\phi}_m(x)$$

The model is subsequently updated by adding the new tree to the previous model:

$$\hat{f}_m(x) = \hat{f}_{m-1}(x) + \hat{p}_m(x)$$

This iterative process is repeated for $m \in \{1,2,\ldots,M\}$, allowing the model to progressively reduce the loss function. The final XGBoost model is therefore expressed as the sum of all weak learners:

$$\hat{f}(x) = \hat{f}_M(x) = \sum_{m=0}^{M} \hat{f}_m(x)$$

Through this additive formulation, XGBoost is capable of modeling complex and non-linear decision boundaries without assuming linear separability of the data. This capability arises from the ensemble of decision trees used in XGBoost, where each tree contributes a piecewise decision rule, resulting in a highly flexible and non-linear decision boundary. Unlike linear models, XGBoost does not assume linear relationships between features and the target variable, nor does it require feature independence. Additionally, due to its tree-based structure, the model is relatively insensitive to feature scaling. The use of second-order information enables efficient and stable optimization, while the ensemble of decision trees captures intricate feature interactions. These characteristics make XGBoost particularly suitable for high-dimensional classification problems, such as music genre classification, where the data exhibits complex and non-linear patterns.

3.4 Supervised Algorithm (FeedForward Neural Network)

Feedforward Neural Network (FNN) is a type of artificial neural network in which Data flows in a single forward direction without loops from the input layer (first layer) through a hidden layer (for simple neural network) or multiple hidden layers to the output layer. It is mainly used for pattern recognition tasks. The output layer uses softmax to produce class probabilities for the 16 genres. The network is trained by minimizing cross-entropy loss between predicted labels and the true genre labels. Regularization (L2), batch normalization, and dropout are used to enhance generalization and prevent overfitting.

- FNN Structure:

It consists of:

1. Input layer: neurons that receive input data. Each neuron is a representation of a feature in data.
2. Hidden layer: one or more layers of neurons between input and output layers. They learn complex patterns in data by applying weighted sum of inputs (bias), and an activation function to introduce nonlinearity into data patterns.
3. Output layer provides final output of the model. It can be either regression (a continuous value) or a classification (a discrete value that can be decoded to get its true label).

- Key Concepts

A neuron's input can be from data like image's pixel numerical values, or it can be an output calculated from another neuron. Each input in a neuron gets a weight that adjusts the input influence the stronger it is the more important it gets. A neuron computation formula is just a simple linear combination; a weighted sum multiplied by features' values with a bias that controls an activation function.

$$z = \sigma(i = 1\sum nwixi + b)$$

An activation function is a method to perform nonlinear decisions into linear combination only when it surpass a strong signal causing the attention to shift towards the strong patterns in data. In music genre classification, the output layer uses a score to compute class's strong prediction by a raw score called (net_i), these scores are added together and transformed into probabilities that sum to 1 using the **softmax function,** the activation function made specifically to fit multiclassification problems. The higher the softmax probability indicates high raw scores. Forward Propagation is the technique used in FNN where data flows in one way from input layers to output only at the end it calculates cross-entropy loss to update models' weight on the next iterations with the help of the adjustable learning rate from gradient descent function. The method in which the FNN feeds back to its weight adjusting it while learning patterns is called Back Propagation.

3.4 Unsupervised Algorithm (K-Means Clustering)

K-Mean Clustering is unsupervised machine learning algorithms, that it learns from input data without labels. K-mean is a centroid-based, partitioning clustering algorithm that group data point into 'K' distinct, non-overlapping group called clusters. Each cluster contains data points with similar characteristics, and each cluster is different from the others. The grouping is done by minimizing the sum of distances between data and corresponding cluster centroid.

sum of the squared Euclidean distances from each point $x_i$to its cluster centroid μ_{c_i}:

$$WCSS = \sum_{j=1}^{K} \sum_{x_i \in c_j} \|x_i - \mu_j\|\text{^}2$$

Where k is the number of clusters, c_j denotes the set of data point assigned to cluster j, x_i is data point, and μ_i represents the centroid of cluster c_j .

K-Means assumes that clusters have a spherical shape, meaning that data points are distributed evenly around the cluster center in all directions. Each cluster is represented by its centroid, which is calculated as the mean of the data points assigned to that cluster. Due to this assumption, K-Means may struggle to correctly identify clusters that are elongated or irregularly shaped.

The algorithm also assumes that all clusters have similar variance, which means that data points are spread around each cluster center in a comparable way. If some clusters are much more spread out than others, K-Means may produce inaccurate groupings.

In addition, K-Means assumes that clusters are relatively similar in size. When clusters differ significantly in the number of data points they contain, larger clusters can dominate the clustering process and affect centroid placement.

However, these assumptions are violated in FMA datasets. Audio features extracted from music signals are typically high-dimensional and complex, non-linear relationships, leading to irregular cluster shapes and overlapping regions.

To address these limitations, K-Means is applied in this study after nonlinear dimensionality reduction using UMAP. By projecting the data into a lower-dimensional space that preserves local similarity, UMAP improves cluster compactness and separability. This transformation makes the distance-based assumptions of K-Means more reasonable and reduces the impact of non-spherical cluster structures.

3.5 Unsupervised Algorithm (HDBSCAN Clustering)

HDBSCAN is the density-based method of clustering HDBSCAN clusters similar data points and separates dissimilar ones depending on how densely the points are distributed. To discover clusters with different densities, sizes, and shapes, it establishes a hierarchical framework. One of its main advantages is its ability to detect and reduce noise by recognizing low-density points that do not belong to any important cluster. By adjusting the minimum cluster size and using a minimal spanning tree, it successfully separates true structure from noise. Because the dataset in this project is extremely overlapping and high-dimensional, HDBSCAN is a great technique for identifying hidden patterns. Suitable for challenging, high-dimensional data where clusters may overlap or have different densities.

Core Distance and Mutual Reachability Distance Definitions

$$d_c(x_p) = d(x_p, x_*)$$

where x_* is the k-th nearest neighbor of point x_p.

$$d_{\text{mreach}}(x_p, x_q) = \max\{d_c(x_p), d_c(x_q), d(x_p, x_q)\}$$

Hierarchical Cluster Tree with Minimum Spanning Tree (MST) A complete graph that is weighted by mutually feasible distances is shown for every point.

Create the minimal spanning tree (MST) to make things simpler.

Also, eliminate edges from the MST in decreasing order of distance to produce a hierarchical cluster tree. Clusters develop and divide with different densities (single linkage with density adjustment).Instead of using distance directly, HDBSCAN uses

$$\lambda = \frac{1}{\text{distance}}$$

High λ leads to low distance, high density. and Low λ leads to low density, high distance.For each cluster the λ_{birth} is value when the cluster appears. And λ_{death} is value when the cluster splits or disappears. For each point p: λ_p is the λ value at which the point leaves the cluster. The stability of a cluster measures its persistence across density levels:

$$\text{stability(cluster)} = \sum_{p \in \text{cluster}} (\lambda_p - \lambda_{\text{birth}})$$

High stability makes a cluster more dependable because it lasts longer across λ levels. Noise or an artifact is likely to result from low stability.

To extract flat clusters in HDBSCAN, all leaf nodes are first assigned as selected clusters. As the hierarchical tree is traversed from leaves to root (reverse topological order), the cluster stabilities are examined at each node. The parent is not chosen, and its stability is transferred to the children if the total of the children's stabilities exceeds the parent's stability; if the parent's stability exceeds the total of its children, the parent is chosen and all its descendants are not chosen. After the root is located, the flat clustering result is the collection of chosen clusters.

Noise is defined as Managing Noise Points that do not belong to any chosen cluster are referred to as noise. HDBSCAN improves stability in noisy datasets by avoiding putting all points into clusters.

Therefore, music tracks and their categories may show different densities due to variations in genre and feature similarity. Because HDBSCAN can handle clusters with varying densities, identify noisy points, and capture intricate patterns in the data, it is a good fit for this situation. So, the approach may find useful groups even when the actual patterns of tracks are complex and overlapping.

3.6 Unsupervised Algorithm (Multistage Clustering (HDBSCAN + K-MEAN))

In this approach, an initial HDBSCAN clustering step to find the main clusters and identify noise according to the density of the data. Then, on the second level, it employs K-Means to further decompose each major cluster into smaller sub-clusters. This two-tier strategy integrates the strengths of density-based structure discovery with partition-based refinement toward better quality and interpretability of clustering.

Stage 1. Initial Structure Discovery using Hierarchical Density-Based Spatial Clustering (HDBSCAN):

The first step includes the identification of global structure in a dataset using HDBSCAN on data. HDBSCAN constructs hierarchical clustering based on the variation in data density and stability at different values of densities. A region that maintains a high density throughout is considered a stable cluster, whereas points falling into low-density areas are explicitly marked as noise or outliers.

This step does not place assumptions on the cluster shape or size and hence finds arbitrarily shaped clusters of varying densities. Explicit handling of noise assures robustness and reliability of the initial partitioning to come from the real structure of the data, rather than from the artefacts caused by sparsity or outliers.

Stage 2. Segmentation using K-Mean:

is applied only to that subset of data points that HDBSCAN confidently assigned to main clusters; noise and unassigned points are excluded. This refinement step divides each main cluster into a predefined number of sub-segments, allowing a more detailed internal structure to be discovered in each density-defined group. Consequently, this phase of the algorithm allows one to reach finer-grained categories and sub-cluster patterns not observable at a coarse-clustering level. Presumably, by increasing the number of the final clusters, the model captures subtle variations in the acoustic feature space, yielding more expressive segmentation aligned with the structure observed in the data.

The multistage clustering framework is based on a set of complementary assumptions that guide how the structure is discovered or refined in the data. First, HDBSCAN assumes that meaningful clusters are regions of high data density surrounded by regions

of low data density. It then assumes the presence of noise or outliers, which the algorithm explicitly identifies and removes for further analysis. This allows the detection of clusters of any shape with variable density, without the imposition of rigid geometrical constraints by looking at density variation across multiple levels.

This process is supported by the density-aware distance measure of HDBSCAN, which captures local neighborhood information. The algorithm, therefore, normalizes the distance relationships across dense and sparse regions, leading to more robust identification of stable cluster structures in complex datasets.

Since K-Means assumes in its second step that meaningful refinements of core clusters provided by HDBSCAN into smaller and more compact sub-clusters are possible, the algorithmic assumptions concerning distances are already more reasonable due to noise-free and pre-structured input data. It also assumes that the minimization of within-cluster variance results in coherent and interpretable segments, making the final clustering suitable for analysis and downstream applications.

This multistep clustering framework is especially well-suited for high-dimensional and noisy datasets, such as the features of music audio, where similarity relationships are complex and often non-linear. Herein, HDBSCAN plays an essential role: to reveal the intrinsic density-based structure of the data and isolate those unreliable or noisy points that do not actually belong to any meaningful cluster. This ensures that the initial grouping reflects natural patterns in the data rather than artifacts caused by noise or outliers.

Afterwards, K-Means refines these density-based clusters into more compact and clearly defined segments. This refinement allows detecting finer-grained patterns of similarity in the data, yielding a larger number of interpretable clusters and better capturing the subtle variations in musical features.

Chapter 4: Methodology

4.1 Data Preprocessing Pipeline

Data preprocessing prepares the dataset for training. The original data had many missing values and overlapping genres. To wrap the techniques used, we merged the metadata and audio features, then dropped columns with more than half missing values. For the genre labels, we used the hierarchal structure in the dataset to fill missing values. As for feature engineering, we created temporal features such as album age and artist profile age. Then we applied scaling methods like StandardScale and implement multiple UMAP algorithms for unsupervised models. Finally, we used Mutual Information to select the most useful features and handled class imbalance with SMOTE and class weights.

- Exploratory Data Analysis EDA

We began by exploring the dataset to understand the features, characteristics, and how they relate to the target genre. During this step, we checked for missing values, studied the diversity of genres, and identified the class imbalance problem.

- Data Preprocessing

We flattened the multi-layered structure of the data to manipulate it easily. Then we merged track metadata with audio features using the track ID. Features with more than 50% missing values were dropped. The target column also had missing values, but we were able to fill them using the genre hierarchy provided in the dataset. We also created temporal features such as album age and artist profile age to capture time trends. Finally, we applied Mutual Information to select the most relevant features that surpasses a defined threshold compiled by how much a feature contributes to the target genres column.

- Data Splits

The dataset was split into training (60%), validation (20%), and testing (20%) sets.

For supervised models the data was split before applying SMOTE and LDA to avoid data leakage. All were performed exclusively on the training set.

- Scaling and Normalization

StandardScaler was used for UMAP and general scaling, while RobustScaler was used to reduce the effect of outliers. Scaling was applied separately to each split to avoid data leakage.

- Label Encoding

Label Encoding methods was used to convert the genre labels into numbers. This allowed the neural network and other models to treat the genre column as a categorical target variable during training assigning each label to a scaler number as 0, 1, …, to cover all classes.

- Uniform Manifold Approximation and Projection (UMAP):

UMAP is a dimensionality reduction algorithm based on manifold learning that projects high-dimensional data into a lower-dimensional space while preserving the essential structure of the data. The algorithm constructs a neighborhood graph to represent local relationships between data points and learns a low-dimensional embedding by minimizing a cross-entropy objective function between the high-dimensional and low-dimensional neighborhood graphs.

4.2 Model Training and Hyperparameter Justification

- **XGBoost:**

The XGBoost algorithm was implemented as a supervised multi-class classification model. Model training was performed using the training subset of the dataset, while the test set was reserved and kept unseen for final evaluation.

Due to the presence of class imbalance in the dataset, SMOTE was applied to the training data prior to model training. This step ensured that minority classes were adequately represented and prevented the learning process from being biased toward majority classes.

Following data balancing, hyperparameter tuning was conducted using GridSearchCV with 3-fold cross-validation. The use of 3-fold cross-validation was chosen to achieve a balance between reliable performance estimation and computational efficiency, given the training cost of XGBoost. Hyperparameter tuning was performed on the SMOTE-enhanced training data to ensure that the selected configuration reflected balanced class learning.

Three experimental training scenarios were subsequently evaluated. First, XGBoost was trained using the original feature set as a baseline model. Second, XGBoost was trained on the SMOTE-balanced data to assess the impact of class balancing. Finally, XGBoost was trained using SMOTE in combination with Linear Discriminant Analysis (LDA) to evaluate the effect of dimensionality reduction on model performance.

The optimized hyperparameters obtained from GridSearchCV were fixed and applied consistently across all three scenarios to ensure a fair and controlled comparison. The selected hyperparameters included the number of estimators, maximum tree depth, learning rate, and subsampling parameters, which collectively regulate model complexity, learning dynamics, and generalization ability.

- **Random Forest Classifier:**

The Random Forest classifier was employed as a robust baseline for this study, utilizing the Bagging (Bootstrap Aggregating) technique. This algorithm constructs a large ensemble of independent decision trees, where each tree is trained on a random subset of the training data. The final prediction is derived through Majority Voting, an approach that effectively mitigates the high variance typically associated with single decision trees and enhances overall predictive stability.

The hyperparameter configuration was carefully fine-tuned to optimize performance and prevent overfitting. The Number of Estimators was increased to 400 to ensure model stability and significantly reduce variance, thereby enhancing prediction reliability without introducing overfitting. To control model complexity, the Max Depth of the trees was capped at 20. This constraint acts as a regularization mechanism, preventing the model from memorizing the training data and forcing it to

learn generalizable patterns. Furthermore, the Min Samples Split was set to 4 to ensure that internal node splits are based on stronger statistical evidence rather than noise. This threshold prevents the trees from capturing granular, non-generalizable details, leading to improved performance on unseen test data. These selected parameters collectively regulate the model's complexity and generalization ability.

- **CatBoost Classifier:**

The CatBoost algorithm was implemented as a supervised multi-class classification model utilizing an advanced gradient boosting approach. Model training was performed using the training subset of the dataset, where decision trees were constructed sequentially. In this iterative process, each new tree attempted to correct the residual errors made by the previous ensemble, progressively minimizing the loss function.

Due to the presence of class imbalance in the dataset, internal Class Weights were applied directly within the model configuration. Inverse weights were assigned based on class frequency—allocating higher weights to rare minority genres and lower weights to frequent genres. This mechanism ensured that minority classes were adequately represented during the cost function calculation and prevented the learning process from being biased toward majority classes.

The hyperparameter configuration was carefully selected to balance model complexity and generalization. The Learning Rate was set to 0.1 to control the magnitude of each tree's contribution, providing a balanced trade-off between convergence speed and the risk of overshooting the optimal solution. The model was constrained to 400 Iterations to prevent the ensemble from becoming overly complex and to mitigate the risk of overfitting while optimizing computational time. A Tree Depth of 6 was utilized, which is the standard efficient depth for CatBoost’s symmetric trees, allowing the model to capture necessary feature interactions without excessive computational cost. Additionally, Verbose logging was enabled every 50 iterations to allow for real-time monitoring of training and validation errors. These selected parameters collectively regulate the model's complexity, learning dynamics, and generalization ability.

- **FNN**:

The Model were trained on tabular features after preprocessing. It has two hidden layers with 128 units (ReLU activation), batch normalization (to feed the next layer of perceptron a normalized sum of inputs avoiding misleading jumps on class weights), L2 regularization (weight decay – a penalty to avoid weights' explosions), and dropout (0.4). The output layer has 16 units with softmax for multi-class classification. We used Adam optimizer with a learning rate of 0.001, batch size 256, and 50 epochs. To handle class imbalance, we tested two setups: class weights only, and SMOTE oversampling on the training data. We also tested training on PCA-reduced features ≈201 components, and on LDA features 15 components, and compared them to FNN model trained on scaled raw features.

- **K-Mean Clustering:**

K-Means was applied as an unsupervised clustering algorithm with the objective of grouping songs based on feature similarity without using labels. Since K-Means relies on distance-based similarity, all numerical features were pre-scaled before training .

Due to the high dimensionality of the FMA dataset, K-Means was not applied directly to the original feature space. Instead, UMAP was used as a preprocessing step to generate a lower-dimensional representation that is more suitable for distance-based clustering. This transformation reduces noise and stabilizes distance relationships, which improves the effectiveness of K-Means.

For this configuration, UMAP was specifically tuned to support K-Means clustering. The neighborhood size was set to n_neighbors = 15, which emphasizes local similarity relationships between data This value provides a balance between capturing meaningful local patterns and avoiding excessive smoothing of the data.

The embedding dimensionality was set to n_components = 2 to project the data into a compact two-dimensional space. This choice simplifies distance computation, reduces noise, and allows clearer cluster separation.

The minimum distance parameter was set to min_dist = 0.0, allowing highly similar data points to be embedded very close to each other.

The cosine distance metric was selected to better capture similarity patterns in high-dimensional acoustic features.A fixed random state was used to ensure reproducibility of the embedding and clustering results.

K-Means clustering was then applied to the UMAP-transformed feature space.The Elbow Method was used on the UMAP embedding to determine an appropriate number of clusters. The number of clusters was evaluated in the range k = 2 to 9, and for each value, K-Means was fitted using n_init = 100 to ensure stable centroid initialization. The corresponding inertia values were recorded and analyzed to assess the trade-off between model complexity and cluster compactness.

Based on the elbow curve, k = 3 was selected as the final number of clusters, as it provided a clear reduction in inertia. Using this value, the final K-Means model was trained on the UMAP-transformed data with n_clusters = 3. The model was fitted using fit_predict, which simultaneously trains the model and assigns a cluster label to each data point.

- **HDBSCAN Clustering:**

The hyperparameters for the model were chosen for handling noise and overlapping areas in the dataset while creating stable, important clusters. Each parameter is important in the building of clusters and the detection of noisy areas because HDBSCAN is sensitive to changes in density. The structure of the dataset, the point distribution, and the goal to create true density-based clusters all shaped the decisions chosen. Additionally, to help improve the separation of areas of high density and improve HDBSCAN in better capturing the fundamental structure of the data prior to clustering, UMAP was used as well as a dimensionality-reducing step.

Before applying the HDBSCAN clustering algorithm, MAP was used for dimensionality reduction prior to the HDBSCAN clustering algorithm. To better capture similarity in the feature space, the data were projected into a 20-dimensional embedding using cosine distance. Compact representations of related information points were kept by using a low minimum distance (min_dist = 0.01) and many

neighbors (n_neighbors = 100) to keep the larger global structure. It also improves the flexibility and efficiency of the following HDBSCAN clustering technique by reducing noise and overlap in the high-dimensional space.

Each hyperparameter that was selected is explained here. min_cluster_size: It is used to indicate how few data points are required to form a cluster. The algorithm will always favor larger and more stable clusters when this parameter is set to 60, which reduces the likelihood of identifying noisy or smaller groups. min_samples: It specifies the minimum number of neighboring points needed for a data point to be called a core point. When this number is set to 10, the model becomes more careful when identifying dense points, increasing the likelihood of recognized clusters while keeping a modest amount of flexibility in areas with lower densities, Metric: Euclidean distance is used to determine how similar two data points are when the distance metric is set to "euclidean." This metric improves a machine's understanding in cluster structure and distances. cluster_selection_method: Clusters are removed from the reduced tree using the (emo) Excess of Mass method. Because it generates more stable and flexible clusters, this approach is recommended for high-dimensional, overlapping datasets with several classes.

- **Multistage Clustering (HDBSCAN + K-MEAN)):**

A multistage unsupervised clustering combining, HDBSCAN, and K-Means, is implemented to handle the noise that characterizes the music feature dataset. The choice of components and hyperparameters was based on empirical testing and their suitability for clustering tasks.

UMAP is used as a preprocessing step to create a representation that is appropriate for density-based and distance-based clustering. The neighborhood size was tuned to n_neighbors = 45 to get broader local structure and make density estimation more stable for HDBSCAN. Embedding dimension is n_components = 15. This is done for reducing dimensionality while keeping important structural information. By setting min_dist = 0.0, similar data points are allowed to form dense regions that are very beneficial in improving cluster stability. Cosine distance is chosen as a better distance metric to represent similarity patterns in high-dimensional acoustic features. For ensuring reproducibility, a fixed random state has been used.

In the first stage of clustering, HDBSCAN was applied to UMAP-reduced data to detect main density-based clusters and noise. The parameter min_cluster_size = 45 ensures that only well-supported clusters are built, while min_samples = 10 controls sensitivity for density estimation. The metric Euclidean distance was chosen since the UMAP embedding provides appropriate space for a distance-based density analysis. The method for choosing clusters was set to leaf for extracting stable clusters from the hierarchical structure. Points classified as noise were excluded from further processing.

In the second step, K-Means was applied only to the data points assigned to clusters by HDBSCAN. To capture internal structure in more detail, every main cluster was further divided into sub-clusters. The underlying assumptions of K-Means now hold as the data at this stage is already structured and noise-reduced. an offset-based labeling approach was used to maintain unique identifiers for the clusters.

4.3Evaluation Setup

4.3.1 Supervised Model

- **Random Forest, CatBoost, and XGBoost Classifiers:**

  The performance was evaluated using the following metrics:

- **Accuracy** is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.
- **Precision** It measures how many of the positive predictions made by the model are correct
- **Recall or Sensitivity** measures how many of the actual positive cases were correctly identified by the model.
- **F1 Score** is the harmonic mean of **precision** and **recall**. It is useful when we need a balance between precision and recall as it combines both into a single number

Model performance was evaluated on both training and testing datasets using accuracy and macro-averaged F1-score. In addition, a classification report and confusion matrix were used on the testing dataset to analyze class-level performance.

- **FNN :**

The FNN was evaluated on the validation set during training and on the test set after training. Each training and validation loss per epoch, accuracy, macro F1, and the confusion matrix were recorded. Macro F1 along with the confusion matrix were used to assess performance across all genres, including minority genres and to ensure majority classes introduce less bias to model's generalization. We compared three input setups: scaled raw features, LDA features, and balancing strategies: class weights only vs. SMOTE.

4.3.2 Unsupervised Models:

- **K-Mean and Multistage Clustering:**

The final setup of the K-Means algorithm was verified using internal validation metrics, specifically the Silhouette Score and the Davies-Bouldin Index, since the problem was unsupervised and no labeled data was required. These metrics were used to compare several setups and validate the final clustering setup.

- **HDBSCAN Clustering:**

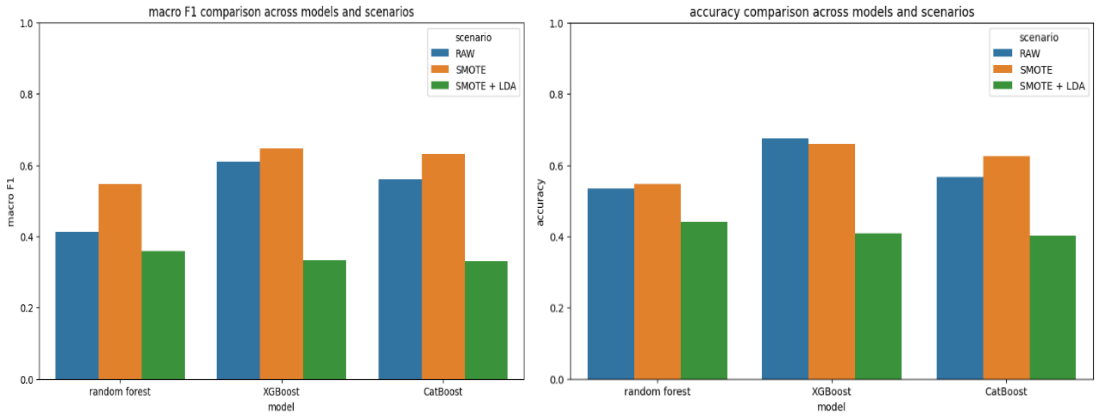
The HDBSCAN clustering algorithm's performance was evaluated using several kinds of quantitative metrics. Higher values indicate more compact and well-separated clusters. The Silhouette Score was used to gauge how well data points fit within their allocated clusters in comparison to other clusters. Also, when available, the agreement between the actual labels and the predicted cluster assignments was evaluated using the Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI). To ensure a fair evaluation that focuses on the stability and consistency of the detected clusters rather than reducing the model for correctly identifying ambiguous or outlier points, data points identified as noise by HDBSCAN.

Chapter 5: Results and Evaluation

5.1 Supervised Model Results

- **Random Forest, CatBoost, and XGBoost Classifiers Results:**

In this section, we present an evaluation of the performance of three supervised classification algorithm models (Random Forest, XGBoost, and CatBoost) across different data processing scenarios: raw data (RAW), data processed using (SMOTE), and data processed using (SMOTE+ LDA). We used accuracy and macro F1-score metrics:



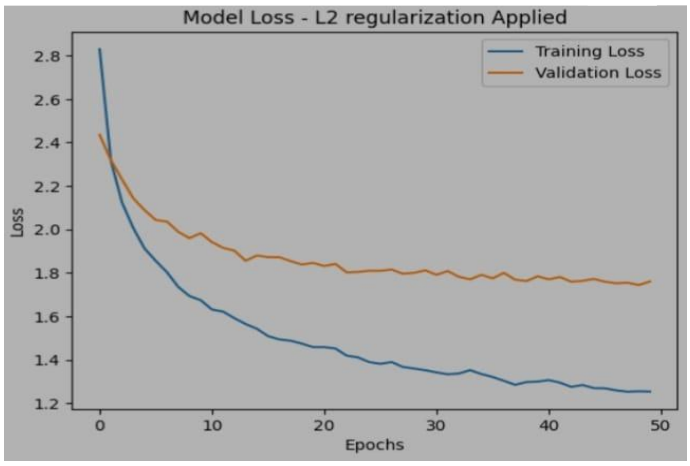
Using SMOTE resulted in a marked improvement in the macro F1 score for all models used.

This indicates that in the first scenario, the raw (RAW) data suffered from an imbalance, and when (SMOTE) was used, it helped the models recognize the few categories significantly, thus raising the F1 value, even if accuracy was slightly affected in some cases. Whereas performance collapsed significantly when using the third scenario (SMOTE +LDA), where accuracy dropped to around 40-43 %and the 35-33 % to F1.This shows that LDA (Linear Analysis) was not suitable for the discriminant analysis of the data. This is because the data were not linearly separable, or dimensionality reduction resulted in the loss of information necessary for classification, causing the models to fail to distinguish between categories.

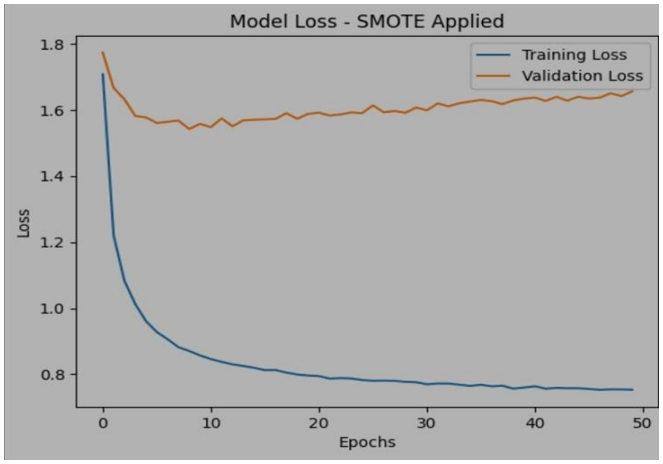
- **FNN Results :**

The feedforward neural network (FNN) was trained on the processed features with different setups: scaled raw data, SMOTE-balanced data, and dimensionality reduction using PCA and LDA. The training accuracy was higher than the validation accuracy, showing that the model learned the training set but struggled to generalize.

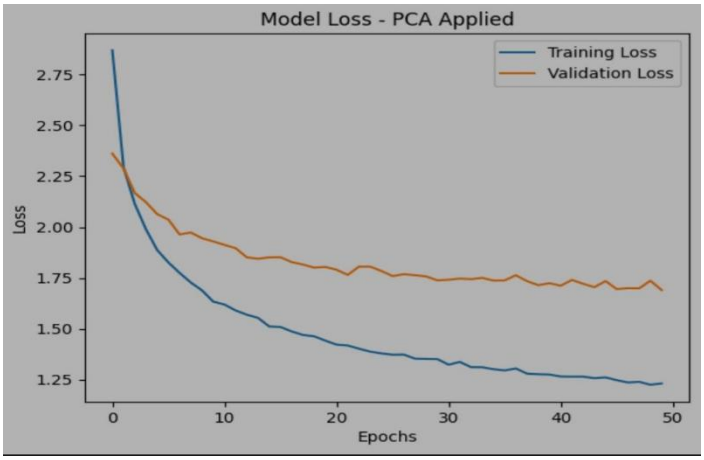
1. Model's losses on scaled data (raw data)



2. Model's losses on SMOTE oversampling applied on data

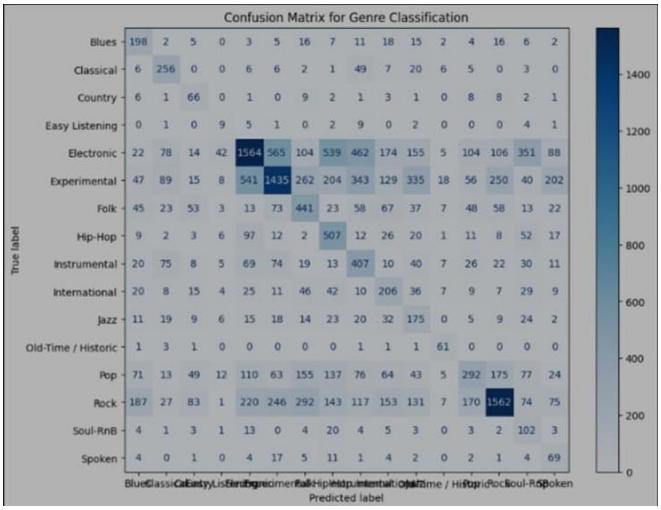


3. Model's losses on PCA features

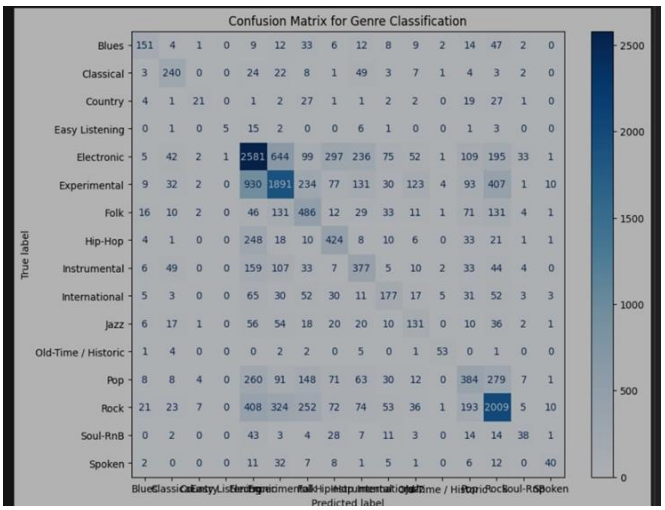


On PCA features, the FNN reached validation accuracy around the mid 40%, with macro F1 scores showing uneven performance across genres. Majority classes were predicted more reliably, while minority genres like Blues and Spoken were often misclassified.

Confusion matrix before applying SMOTE oversampling



Confusion matrix after applying SMOTE oversampling



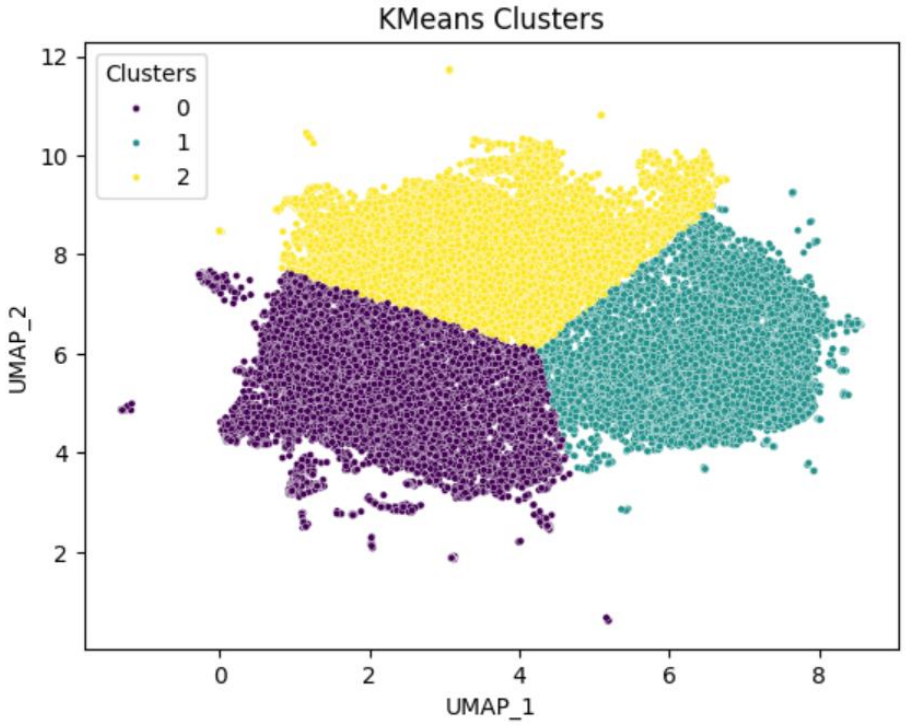
The confusion matrix emphasised that the FNN outperformed on majority genres, while minority classes were difficult to classify.

Using SMOTE improved balance in predictions but did not raise validation accuracy significantly, suggesting that new samples did not fully match the real distribution.

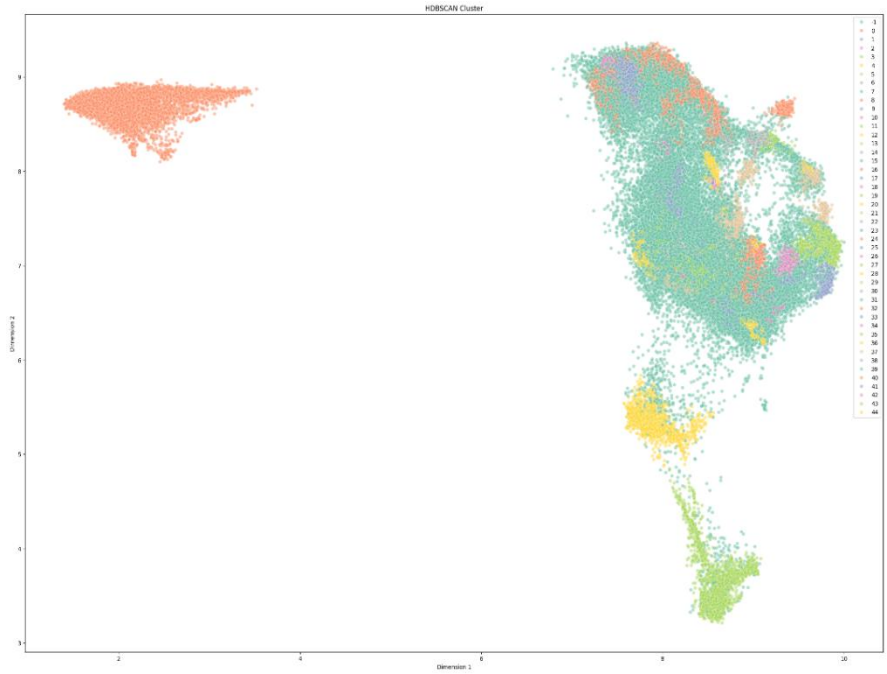
the FNN provided useful insights as a neural approach, however, tree-based models achieved stronger generalization on this dataset.

5.2 Unsupervised Model Results

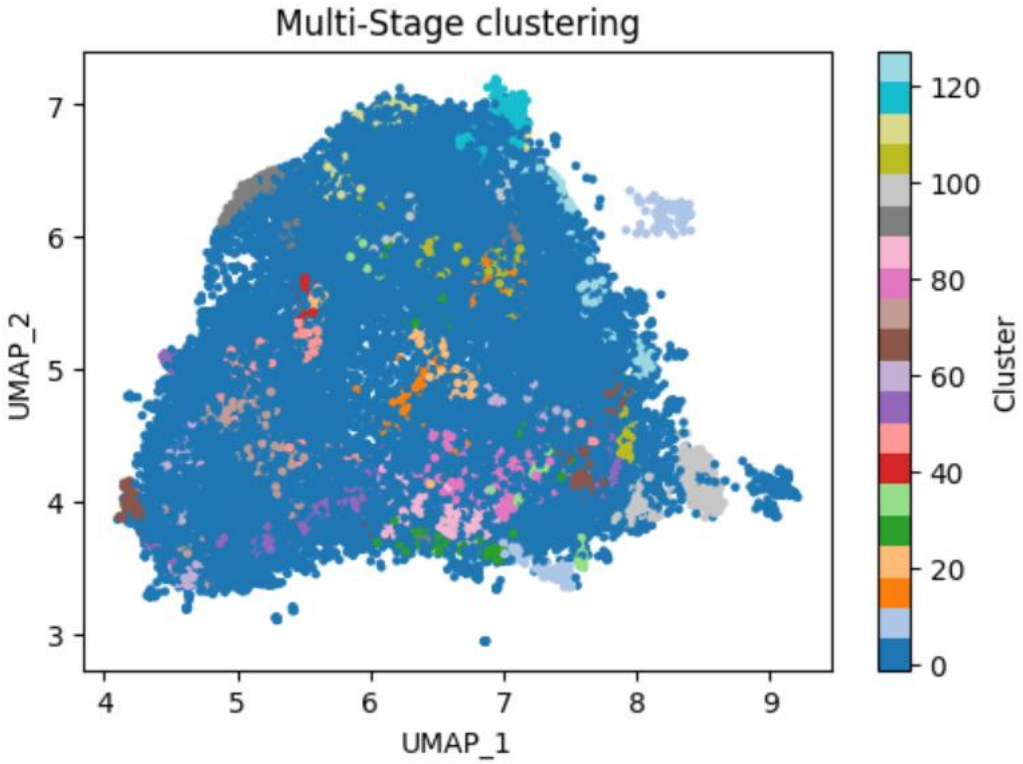
- **K-Mean Clustering Visualization:**



- **HDBSCAN Clustering Visualization:**



- **Multisatge Clustering Visualization:**



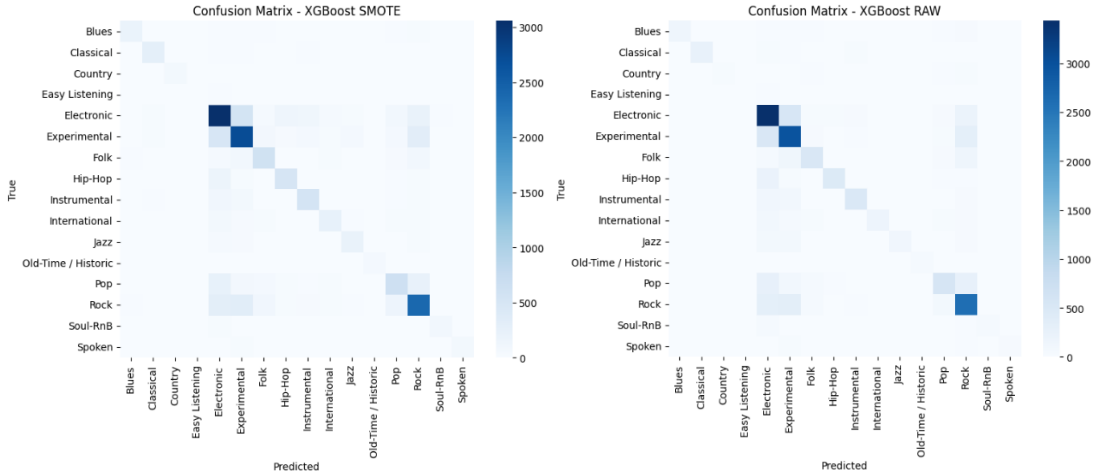
5.3 Discussion of Performance and trade-offs

- **Supervised algorithms**

When comparing supervised models based on accuracy and macro F-1, we find that:

	model	scenario	accuracy	macro F1
0	random forest	RAW	0.534222	0.413562
1	random forest	SMOTE	0.546544	0.547717
2	random forest	SMOTE + LDA	0.441134	0.359107
3	XGBoost	RAW	0.674975	0.609729
4	XGBoost	SMOTE	0.659236	0.646917
5	XGBoost	SMOTE + LDA	0.408368	0.332021
6	CatBoost	RAW	0.565476	0.560100
7	CatBoost	SMOTE	0.625182	0.630713
8	CatBoost	SMOTE + LDA	0.402879	0.330688

Best performance of the model XGBoost at accuracy and Macro F1-score



In terms of results, the XGBoost model performed best overall, achieving the highest Accuracy of 67.3% with RAW data and a peak Macro F1-score of 64.9% using SMOTE. CatBoost ranked second, showing significant improvement with SMOTE and reaching an F1-score of 63.8%. Although Random Forest was the lowest-performing model, it exhibited a notable performance boost with oversampling, where its F1-score jumped from 0.41 RAW to 0.55 SMOTE.

- **Unsupervised algorithms**

Model	Silhouette Score
K-mean	0.424
HDBSCAN	0.509
Multistage Clustering (HBDSCN+ K-Mean)	0.428

When comparing unsupervised models based on silhouettes score, we find that:

The HDBSCAN model performed the best among all evaluated approaches, indicating stronger group separation compared to the other models. This suggests that density-based clustering is more effective at capturing the underlying structure of the data.

In contrast, K-Means exhibited weaker group separation, which is expected given the non-linear and complex nature of the dataset and the spherical cluster assumption of K-Means.

The multistage clustering model (HDBSCAN + K-Means) obtained a lower separation score, indicating that increasing the number of clusters through further subdivision did not improve group separation. Instead, the refinement step weakened the original density-based structure identified by HDBSCAN, resulting in reduced cluster quality.

These results show that while multi-stage clustering may sometimes generate more detailed groupings, in this case it decreased cluster quality by changing the HDBSCAN-identified natural structure. Weaker separation resulted from the hard geometric limitations enforced by K-Means' refinement stage, which met with the flexible density-based clusters. This shows that preserving the initial density-based clusters is more efficient than further subdividing them for data with overlapping patterns and different densities. All things considered, HDBSCAN by itself achieves the optimal balance between handling noisy points in the dataset, stability, and useful separation.

Chapter 6: Discussion and Conclusion

6.1 Interpretation of findings and Implications

Although we initially expected LDA to achieve good class separation, the results did not meet these expectations, mainly due to class overlap and the non-linear structure of the dataset.

Although PCA features and LDA features in FNN model were close on performance, and that is due to the activation function ability to provide nonlinearity capturing complex patterns through the layers. According to FMA paper, one possible approach is to train CNN model on audio features in a wave images representation.

6.2 Potential Improvements and Extensions

Future work could explore deep learning models based on spectrogram representations and ensemble methods by combining multiple models to further improve classification performance.

One of the findings from the feedforward neural network experiments was a hybrid solution to handle class imbalance. The results in the confusion matrix above showed that oversampling with SMOTE is most useful for the smallest minority classes, while the other classes can be handled using class weights. This combination can help the network balance predictions across genres.

6.3 Conclusion

This study investigated music genre classification using machine learning techniques. The results show that music data is complex and contains overlapping genres, which affects classification performance. Overall, the applied models provided reasonable results for this task.

References

[1] The theoretical background and mathematical formulation of the XGBoost model presented in the theoretical Background section are based on the following source: https://medium.com/hoyalytics/xgboost-theory-and-application-4801a5dba4fb

[2] The theoretical background and mathematical formulation of the HDBSCAN model are based on the following source: https://hdbscan.readthedocs.io/en/latest/how_hdbscan_works.html

[3] https://scikit-learn.org/stable/modules/generated/sklearn.cluster.HDBSCAN.html

[4] https://medium.com/@sanyagubrani/k-means-clustering-algorithm-159a18d07202

[5] https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/

[6] https://pubmed.ncbi.nlm.nih.gov/33057332/

[7] https://www.datacamp.com/tutorial/understanding-umap-guide-to-dimensionality-reduction

[8] https://www.ibm.com/think/topics/linear-discriminant-analysis

[9] The theoretical background and mathematical formulation of the CatBoost model presented in the theoretical Background section are based on the following source: https://www.geeksforgeeks.org/machine-learning/catboost-ml/

[10] These papers were reviewed to help interpret our results and understand whether they fall within an expected range

https://arxiv.org/pdf/2509.01762

https://arxiv.org/pdf/1804.01149

[11]https://medium.com/@ujangriswanto08/step-by-step-implementation-of-hdbscan-in-python-or-r-94350202c7c1

[12] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in Proc. 18th Int. Soc. Music Information Retrieval Conf. (ISMIR), 2017. mdeff/fma: FMA: A Dataset For Music Analysis mdeff/fma: FMA: A Dataset For Music Analysis

[13] Music Genre Classification using Machine Learning Techniques Hareesh Bahuleyan University of Waterloo, ON, Canada. 1804.01149 HareeshBahuleyan/music-genre-classification: Recognizing the genre of music files using machine learning and deep learning models